

1. Arrays (5 Questions)

1. Find the largest element in an array.
2. Find the second largest element without sorting.
3. Remove duplicate elements from a sorted array.
4. Rotate an array to the right by **k** positions.
5. Find the maximum sum subarray using **Kadane's Algorithm**.

1. Find the Largest Element in an Array

```
public class LargestElement {  
    public static void main(String[] args) {  
  
        int[] arr = {12, 45, 7, 89, 23};  
  
        int largest = arr[0];  
  
        for (int i = 1; i < arr.length; i++) {  
            if (arr[i] > largest) {  
                largest = arr[i];  
            }  
        }  
  
        System.out.println("Largest Element = " + largest);  
    }  
}
```

Output

Largest Element = 89

2. Find the Second Largest Element Without Sorting

```
public class SecondLargest {  
    public static void main(String[] args) {  
  
        int[] arr = {10, 25, 8, 55, 42};
```

```

int largest = Integer.MIN_VALUE;
int secondLargest = Integer.MIN_VALUE;

for (int num : arr) {

    if (num > largest) {
        secondLargest = largest;
        largest = num;
    } else if (num > secondLargest && num != largest) {
        secondLargest = num;
    }
}

System.out.println("Second Largest = " + secondLargest);
}
}

```

Output

Second Largest = 42

Time Complexity: $O(n)$

3. Remove Duplicate Elements from a Sorted Array

```

public class RemoveDuplicates {
    public static void main(String[] args) {

        int[] arr = {1,1,2,2,3,4,4,5};

        int j = 0;

        for (int i = 1; i < arr.length; i++) {

            if (arr[i] != arr[j]) {
                j++;
                arr[j] = arr[i];
            }
        }
    }
}

```

```
        System.out.println("Array after removing duplicates:");

        for (int i = 0; i <= j; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

Output

1 2 3 4 5

Time Complexity: $O(n)$

4. Rotate an Array to the Right by K Positions

```
public class RotateArray {

    public static void reverse(int[] arr, int start, int end) {

        while (start < end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;

            start++;
            end--;
        }
    }

    public static void main(String[] args) {

        int[] arr = {1,2,3,4,5,6,7};

        int k = 3;

        k = k % arr.length;

        reverse(arr, 0, arr.length - 1);
    }
}
```

```
reverse(arr, 0, k - 1);
reverse(arr, k, arr.length - 1);

for (int num : arr) {
    System.out.print(num + " ");
}
}
```

Output

5 6 7 1 2 3 4

Time Complexity: $O(n)$

5. Maximum Sum Subarray (Kadane's Algorithm)

```
public class KadaneAlgorithm {
    public static void main(String[] args) {

        int[] arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};

        int currentSum = arr[0];
        int maxSum = arr[0];

        for (int i = 1; i < arr.length; i++) {

            currentSum = Math.max(arr[i], currentSum + arr[i]);
            maxSum = Math.max(maxSum, currentSum);
        }

        System.out.println("Maximum Sum = " + maxSum);
    }
}
```

Output

Maximum Sum = 6

The maximum sum subarray is:

[4, -1, 2, 1]

2. Strings (5 Questions)

1. Reverse a string.
2. Check whether a string is a palindrome.
3. Count vowels and consonants.
4. Find the first non-repeating character.
5. Check whether two strings are anagrams.

1. Reverse a String

```
public class ReverseString {  
    public static void main(String[] args) {  
  
        String str = "Hello";  
        String reverse = "";  
  
        for (int i = str.length() - 1; i >= 0; i--) {  
            reverse += str.charAt(i);  
        }  
  
        System.out.println("Original String: " + str);  
        System.out.println("Reversed String: " + reverse);  
    }  
}
```

Output

Original String: Hello

Reversed String: olleH

2. Check Whether a String is a Palindrome

```
public class Palindrome {  
    public static void main(String[] args) {  
  
        String str = "madam";  
        boolean isPalindrome = true;  
  
        for (int i = 0; i < str.length() / 2; i++) {  
            if (str.charAt(i) != str.charAt(str.length() - 1 - i)) {  
                isPalindrome = false;  
                break;  
            }  
        }  
  
        if (isPalindrome)  
            System.out.println("Palindrome");  
        else  
            System.out.println("Not Palindrome");  
    }  
}
```

Output

Palindrome

3. Count Vowels and Consonants

```
public class CountVowelsConsonants {  
    public static void main(String[] args) {  
  
        String str = "Java Programming";  
        str = str.toLowerCase();  
  
        int vowels = 0;  
        int consonants = 0;  
  
        for (int i = 0; i < str.length(); i++) {  
  
            char ch = str.charAt(i);  
  
            if (ch >= 'a' && ch <= 'z') {  
  
                if (ch == 'a' || ch == 'e' || ch == 'i'
```

```

        || ch == 'o' || ch == 'u') {
            vowels++;
        } else {
            consonants++;
        }
    }
}

System.out.println("Vowels = " + vowels);
System.out.println("Consonants = " + consonants);
}
}

```

Output

Vowels = 5
Consonants = 10

4. Find the First Non-Repeating Character

```
import java.util.HashMap;
```

```

public class FirstNonRepeatingCharacter {
    public static void main(String[] args) {

        String str = "aabbcdde";

        HashMap<Character, Integer> map = new HashMap<>();

        // Count frequency
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        // Find first non-repeating character
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);

            if (map.get(ch) == 1) {
                System.out.println("First Non-Repeating Character: " + ch);
                break;
            }
        }
    }
}

```

```
    }  
  }  
}
```

Output

First Non-Repeating Character: c

5. Check Whether Two Strings are Anagrams

```
public class Anagram {  
    public static void main(String[] args) {  
  
        String str1 = "listen";  
        String str2 = "silent";  
  
        if (str1.length() != str2.length()) {  
            System.out.println("Not Anagram");  
            return;  
        }  
  
        int[] count = new int[26];  
  
        for (int i = 0; i < str1.length(); i++) {  
            count[str1.charAt(i) - 'a']++;  
            count[str2.charAt(i) - 'a']--;  
        }  
  
        boolean isAnagram = true;  
  
        for (int i = 0; i < 26; i++) {  
            if (count[i] != 0) {  
                isAnagram = false;  
                break;  
            }  
        }  
  
        if (isAnagram)  
            System.out.println("Anagram");  
        else  
            System.out.println("Not Anagram");  
    }  
}
```


Output

Anagram

3. Searching (5 Questions)

1. Linear Search.
2. Binary Search.
3. Find the first occurrence of an element.
4. Find the last occurrence of an element.
5. Search an element in a rotated sorted array.

1. Linear Search

```
public class LinearSearch {  
    public static void main(String[] args) {  
  
        int[] arr = {10, 25, 30, 45, 60};  
        int target = 45;  
  
        int index = -1;  
  
        for (int i = 0; i < arr.length; i++) {  
  
            if (arr[i] == target) {  
                index = i;  
                break;  
            }  
        }  
  
        if (index != -1)  
            System.out.println("Element found at index " + index);  
        else  
            System.out.println("Element not found");  
    }  
}
```

Output

Element found at index 3

Time Complexity: $O(n)$

2. Binary Search

Note: Binary Search works only on a **sorted array**.

```
public class BinarySearch {
    public static void main(String[] args) {

        int[] arr = {10, 20, 30, 40, 50, 60, 70};
        int target = 50;

        int low = 0;
        int high = arr.length - 1;
        int index = -1;

        while (low <= high) {

            int mid = low + (high - low) / 2;

            if (arr[mid] == target) {
                index = mid;
                break;
            }
            else if (arr[mid] < target) {
                low = mid + 1;
            }
            else {
                high = mid - 1;
            }
        }

        if (index != -1)
            System.out.println("Element found at index " + index);
        else
            System.out.println("Element not found");
    }
}
```

```
}
```

Output

Element found at index 4

Time Complexity: $O(\log n)$

3. Find the First Occurrence of an Element

```
public class FirstOccurrence {
    public static void main(String[] args) {

        int[] arr = {2, 4, 4, 4, 6, 8, 10};
        int target = 4;

        int low = 0;
        int high = arr.length - 1;
        int answer = -1;

        while (low <= high) {

            int mid = low + (high - low) / 2;

            if (arr[mid] == target) {
                answer = mid;
                high = mid - 1;
            }
            else if (arr[mid] < target) {
                low = mid + 1;
            }
            else {
                high = mid - 1;
            }
        }

        System.out.println("First Occurrence = " + answer);
    }
}
```

Output

First Occurrence = 1

Time Complexity: $O(\log n)$

4. Find the Last Occurrence of an Element

```
public class LastOccurrence {
    public static void main(String[] args) {

        int[] arr = {2, 4, 4, 4, 6, 8, 10};
        int target = 4;

        int low = 0;
        int high = arr.length - 1;
        int answer = -1;

        while (low <= high) {

            int mid = low + (high - low) / 2;

            if (arr[mid] == target) {
                answer = mid;
                low = mid + 1;
            }
            else if (arr[mid] < target) {
                low = mid + 1;
            }
            else {
                high = mid - 1;
            }
        }

        System.out.println("Last Occurrence = " + answer);
    }
}
```

Output

Last Occurrence = 3

Time Complexity: $O(\log n)$

5. Search an Element in a Rotated Sorted Array

```
public class SearchRotatedArray {
    public static void main(String[] args) {

        int[] arr = {4, 5, 6, 7, 0, 1, 2};
        int target = 0;

        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {

            int mid = low + (high - low) / 2;

            if (arr[mid] == target) {
                System.out.println("Element found at index " + mid);
                return;
            }

            // Left half is sorted
            if (arr[low] <= arr[mid]) {

                if (target >= arr[low] && target < arr[mid]) {
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            }

            // Right half is sorted
            else {

                if (target > arr[mid] && target <= arr[high]) {
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
        }
    }
}
```

```
    }  
    System.out.println("Element not found");  
}  
}
```

Output

Element found at index 4

4. Sorting (5 Questions)

1. Bubble Sort.
2. Selection Sort.
3. Insertion Sort.
4. Merge Sort.
5. Quick Sort.

1. Bubble Sort

Idea: Repeatedly swap adjacent elements if they are in the wrong order.

```
public class BubbleSort {  
  
    public static void main(String[] args) {  
  
        int[] arr = {5, 3, 8, 4, 2};
```

```
for (int i = 0; i < arr.length - 1; i++) {  
  
    for (int j = 0; j < arr.length - 1 - i; j++) {  
  
        if (arr[j] > arr[j + 1]) {  
  
            int temp = arr[j];  
            arr[j] = arr[j + 1];  
            arr[j + 1] = temp;  
        }  
    }  
}
```

```
System.out.print("Sorted Array: ");
```

```
for (int num : arr)  
    System.out.print(num + " ");  
  
}
```

Output

Sorted Array: 2 3 4 5 8

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

2. Selection Sort

Idea: Find the minimum element and place it at the beginning.

```
public class SelectionSort {  
  
    public static void main(String[] args) {  
  
        int[] arr = {64, 25, 12, 22, 11};  
  
        for (int i = 0; i < arr.length - 1; i++) {  
  
            int minIndex = i;  
  
            for (int j = i + 1; j < arr.length; j++) {  
  
                if (arr[j] < arr[minIndex]) {  
                    minIndex = j;  
                }  
            }  
  
            int temp = arr[i];  
            arr[i] = arr[minIndex];  
            arr[minIndex] = temp;  
        }  
    }  
}
```



```
        System.out.print("Sorted Array: ");

        for (int num : arr)

            System.out.print(num + " ");

    }
}
```

Output

Sorted Array: 11 12 22 25 64

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

3. Insertion Sort

Idea: Insert each element into its correct position in the sorted part.

```
public class InsertionSort {

    public static void main(String[] args) {

        int[] arr = {12, 11, 13, 5, 6};

        for (int i = 1; i < arr.length; i++) {

            int key = arr[i];

            int j = i - 1;
```

```

while (j >= 0 && arr[j] > key) {

    arr[j + 1] = arr[j];

    j--;

}

arr[j + 1] = key;

}

System.out.print("Sorted Array: ");

for (int num : arr)

    System.out.print(num + " ");

}

```

Output

Sorted Array: 5 6 11 12 13

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

4. Merge Sort

Idea: Divide the array into two halves, sort each half, then merge them.

```
public class MergeSort {

    static void merge(int[] arr, int left, int mid, int right) {

        int n1 = mid - left + 1;

        int n2 = right - mid;

        int[] L = new int[n1];

        int[] R = new int[n2];

        for (int i = 0; i < n1; i++)

            L[i] = arr[left + i];

        for (int j = 0; j < n2; j++)

            R[j] = arr[mid + 1 + j];

        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {

            if (L[i] <= R[j])

                arr[k++] = L[i++];

            else

                arr[k++] = R[j++];

        }

    }

}
```

```
}
```

```
while (i < n1)
```

```
    arr[k++] = L[i++];
```

```
while (j < n2)
```

```
    arr[k++] = R[j++];
```

```
}
```

```
static void mergeSort(int[] arr, int left, int right) {
```

```
    if (left < right) {
```

```
        int mid = (left + right) / 2;
```

```
        mergeSort(arr, left, mid);
```

```
        mergeSort(arr, mid + 1, right);
```

```
        merge(arr, left, mid, right);
```

```
    }
```

```
}
```

```
public static void main(String[] args) {
```

```
int[] arr = {38, 27, 43, 3, 9, 82, 10};

mergeSort(arr, 0, arr.length - 1);

System.out.print("Sorted Array: ");

for (int num : arr)
    System.out.print(num + " ");
}
```

Output

Sorted Array: 3 9 10 27 38 43 82

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

5. Quick Sort

Idea: Choose a pivot, place it in the correct position, and recursively sort both sides.

```
public class QuickSort {

    static int partition(int[] arr, int low, int high) {

        int pivot = arr[high];

        int i = low - 1;
```

```
for (int j = low; j < high; j++) {
```

```
    if (arr[j] < pivot) {
```

```
        i++;
```

```
        int temp = arr[i];
```

```
        arr[i] = arr[j];
```

```
        arr[j] = temp;
```

```
    }
```

```
}
```

```
int temp = arr[i + 1];
```

```
arr[i + 1] = arr[high];
```

```
arr[high] = temp;
```

```
return i + 1;
```

```
}
```

```
static void quickSort(int[] arr, int low, int high) {
```

```
    if (low < high) {
```

```

        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);

        quickSort(arr, pi + 1, high);
    }
}

```

```

public static void main(String[] args) {

```

```

    int[] arr = {10, 7, 8, 9, 1, 5};

```

```

    quickSort(arr, 0, arr.length - 1);

```

```

    System.out.print("Sorted Array: ");

```

```

    for (int num : arr)

```

```

        System.out.print(num + " ");

```

```

    }

```

```

}

```

Output

Sorted Array: 1 5 7 8 9 10

5. Stack (5 Questions)

1. Implement Stack using Array.
2. Reverse a string using Stack.
3. Check balanced parentheses.
4. Find Next Greater Element.
5. Implement Min Stack.

1. Implement Stack using Collection Framework

```
import java.util.Stack;
```

```
public class StackExample {
```

```
    public static void main(String[] args) {
```

```
        Stack<Integer> stack = new Stack<>();
```

```
        // Push
```

```
        stack.push(10);
```

```
        stack.push(20);
```

```
        stack.push(30);
```

```
        System.out.println("Stack : " + stack);
```

```
        // Peek
```

```
        System.out.println("Top Element : " + stack.peek());
```

```
        // Pop
```



```
System.out.println("Removed : " + stack.pop());
```

```
System.out.println("Stack After Pop : " + stack);
```

```
System.out.println("Is Empty : " + stack.isEmpty());
```

```
System.out.println("Size : " + stack.size());
```

```
}
```

```
}
```

Output

Stack : [10, 20, 30]

Top Element : 30

Removed : 30

Stack After Pop : [10, 20]

Is Empty : false

Size : 2

Time Complexity

- $\text{push()} \rightarrow O(1)$
 - $\text{pop()} \rightarrow O(1)$
 - $\text{peek()} \rightarrow O(1)$
-

2. Reverse a String using Stack

```
import java.util.Stack;

public class ReverseString {

    public static void main(String[] args) {

        String str = "HELLO";

        Stack<Character> stack = new Stack<>();

        for(char ch : str.toCharArray()){

            stack.push(ch);

        }

        String reverse = "";

        while(!stack.isEmpty()){

            reverse += stack.pop();

        }

        System.out.println("Original : " + str);

        System.out.println("Reverse : " + reverse);

    }

}
```

```
}  
}
```

Output

Original : HELLO

Reverse : OLLEH

Time Complexity: $O(n)$

3. Check Balanced Parentheses

```
import java.util.Stack;
```

```
public class BalancedParentheses {
```

```
    public static void main(String[] args) {
```

```
        String str = "{[()]}"
```

```
        Stack<Character> stack = new Stack<>();
```

```
        boolean balanced = true;
```

```
        for(char ch : str.toCharArray()){
```

```
            if(ch=='(' || ch=='{' || ch=='['){
```

```
        stack.push(ch);
    }

    else{

        if(stack.isEmpty()){

            balanced = false;

            break;

        }

        char top = stack.pop();

        if((ch=='') && top!='(') ||

            (ch=='}') && top!='{' ||

            (ch==''] && top!='[')){

            balanced = false;

            break;

        }

    }

}

if(!stack.isEmpty())

    balanced = false;
```

```
    if(balanced)

        System.out.println("Balanced");

    else

        System.out.println("Not Balanced");

}

}
```

Output

Balanced

Time Complexity: $O(n)$

4. Next Greater Element

```
import java.util.Stack;

public class NextGreaterElement {

    public static void main(String[] args) {

        int[] arr = {4, 5, 2, 25};

        Stack<Integer> stack = new Stack<>();

        int[] ans = new int[arr.length];
```

```
for(int i = arr.length-1; i>=0; i--){

    while(!stack.isEmpty() && stack.peek()<=arr[i]){

        stack.pop();

    }

    if(stack.isEmpty())

        ans[i] = -1;

    else

        ans[i] = stack.peek();

    stack.push(arr[i]);

}

for(int i=0;i<arr.length;i++){

    System.out.println(arr[i]+" --> "+ans[i]);

}

}
```

Output

4 --> 5

5 --> 25

2 --> 25

25 --> -1

Time Complexity: $O(n)$

5. Implement Min Stack

```
import java.util.Stack;

public class MinStack {

    Stack<Integer> stack = new Stack<>();

    Stack<Integer> minStack = new Stack<>();

    public void push(int x){

        stack.push(x);

        if(minStack.isEmpty() || x<=minStack.peek()){

            minStack.push(x);

        }

    }

    public void pop(){

        if(stack.peek().equals(minStack.peek())){

            minStack.pop();
```

```
}
```

```
stack.pop();
```

```
}
```

```
public int getMin(){
```

```
    return minStack.peek();
```

```
}
```

```
public static void main(String[] args) {
```

```
    MinStack ms = new MinStack();
```

```
    ms.push(5);
```

```
    ms.push(2);
```

```
    ms.push(10);
```

```
    ms.push(1);
```

```
    System.out.println("Minimum : " + ms.getMin());
```

```
    ms.pop();
```

```
    System.out.println("Minimum After Pop : " + ms.getMin());
```

```
}
```



```
}
```

Output

Minimum : 1

Minimum After Pop : 2

6. Queue (5 Questions)

1. Implement Queue using Array.
2. Implement Circular Queue.
3. Implement Queue using Two Stacks.
4. Reverse the first K elements of a Queue.
5. Generate Binary Numbers from 1 to N using Queue.

1. Implement Queue using Collection Framework

```
import java.util.*;
```

```
public class QueueExample {
```

```
    public static void main(String[] args) {
```

```
        Queue<Integer> queue = new LinkedList<>();
```

```
        // Enqueue
```

```
        queue.offer(10);
```

```
        queue.offer(20);
```

```
queue.offer(30);

System.out.println("Queue : " + queue);

// Front Element
System.out.println("Front : " + queue.peek());

// Dequeue
System.out.println("Removed : " + queue.poll());

System.out.println("Queue After Removal : " + queue);

System.out.println("Size : " + queue.size());

System.out.println("Is Empty : " + queue.isEmpty());
}
}
```

Output

Queue : [10, 20, 30]

Front : 10

Removed : 10

Queue After Removal : [20, 30]

Size : 2

Is Empty : false

Time Complexity

- `offer()` → $O(1)$
 - `poll()` → $O(1)$
 - `peek()` → $O(1)$
-

2. Implement Circular Queue

Using **ArrayDeque** (Circular Queue Implementation)

```
import java.util.*;

public class CircularQueueExample {

    public static void main(String[] args) {

        ArrayDeque<Integer> queue = new ArrayDeque<>();

        queue.offer(10);
        queue.offer(20);
        queue.offer(30);

        System.out.println(queue);

        queue.poll();

        queue.offer(40);
```

```
System.out.println(queue);
```

```
System.out.println("Front : " + queue.peek());
```

```
System.out.println("Rear : " + queue.peekLast());
```

```
}
```

```
}
```

Output

[10, 20, 30]

[20, 30, 40]

Front : 20

Rear : 40

Time Complexity: $O(1)$

3. Implement Queue using Two Stacks

```
import java.util.*;
```

```
public class QueueUsingTwoStacks {
```

```
    static Stack<Integer> s1 = new Stack<>();
```

```
    static Stack<Integer> s2 = new Stack<>();
```

```
static void enqueue(int x) {  
    s1.push(x);  
}
```

```
static int dequeue() {
```

```
    if (s2.isEmpty()) {
```

```
        while (!s1.isEmpty()) {
```

```
            s2.push(s1.pop());
```

```
        }
```

```
    }
```

```
    return s2.pop();
```

```
}
```

```
public static void main(String[] args) {
```

```
    enqueue(10);
```

```
    enqueue(20);
```

```
    enqueue(30);
```

```
    System.out.println(dequeue());
```

```
enqueue(40);
```

```
System.out.println(dequeue());
```

```
System.out.println(dequeue());
```

```
}
```

```
}
```

Output

10

20

30

Time Complexity

- Enqueue $\rightarrow O(1)$
- Dequeue $\rightarrow$ Amortized $O(1)$

4. Reverse the First K Elements of a Queue

```
import java.util.*;
```

```
public class ReverseFirstK {
```

```
    public static void main(String[] args) {
```

```
        Queue<Integer> queue = new LinkedList<>();
```

```
queue.offer(10);
```

```
queue.offer(20);
```

```
queue.offer(30);
```

```
queue.offer(40);
```

```
queue.offer(50);
```

```
int k = 3;
```

```
Stack<Integer> stack = new Stack<>();
```

```
for(int i=0;i<k;i++){
```

```
    stack.push(queue.poll());
```

```
}
```

```
while(!stack.isEmpty()){
```

```
    queue.offer(stack.pop());
```

```
}
```

```
int size = queue.size();
```

```
for(int i=0;i<size-k;i++){
```

```
    queue.offer(queue.poll());
```

```
}
```

```
        System.out.println(queue);
    }
}
```

Output

[30, 20, 10, 40, 50]

Time Complexity: $O(n)$

5. Generate Binary Numbers from 1 to N using Queue

```
import java.util.*;

public class BinaryNumbers {

    public static void main(String[] args) {

        int n = 10;

        Queue<String> queue = new LinkedList<>();

        queue.offer("1");

        for(int i=1;i<=n;i++){
```



```
String current = queue.poll();
```

```
System.out.println(current);
```

```
queue.offer(current + "0");
```

```
queue.offer(current + "1");
```

```
}
```

```
}
```

```
}
```

Output

1

10

11

100

101

110

111

1000

1001

1010

Time Complexity: $O(n)$

7. Linked List (5 Questions)

1. Insert a node at the beginning.
2. Delete a node by value.
3. Reverse a Linked List.
4. Find the middle node.
5. Detect a cycle using Floyd's Algorithm.

8. Trees (5 Questions)

1. Inorder Traversal.
2. Preorder Traversal.
3. Postorder Traversal.
4. Level Order Traversal.
5. Find the height of a Binary Tree.

9. Binary Search Tree (BST) (5 Questions)

1. Insert a node.
2. Search a node.
3. Delete a node.
4. Find Minimum and Maximum.
5. Validate a Binary Search Tree.

10. Heap (5 Questions)

1. Implement Max Heap.
2. Implement Min Heap.
3. Heap Sort.
4. Find K Largest Elements.
5. Find Kth Smallest Element.

11. Recursion (5 Questions)

1. Factorial.
2. Fibonacci Series.
3. Power (x^n).
4. Sum of digits.
5. Tower of Hanoi.

12. Dynamic Programming (5 Questions)

1. Climbing Stairs.
2. House Robber.
3. 0/1 Knapsack.
4. Longest Common Subsequence.
5. Coin Change.